SRGAN midterm Cats versus dogs in Google Colab

Goal: compare MobileNetV2 classifier A trained on real 128 x 128 images with classifier B trained on SRGAN reconstructions of the same training images. The generator learns **32 x 32 RGB -> 128 x 128 RGB**, not random-noise generation.

Before running: select Runtime > Change runtime type > GPU. In Section 2, set `DATA_SOURCE` to your Assignment 1 ZIP or image folder in Drive. Keep the original dataset; this notebook rebuilds the required split instead of reusing Assignment 1's old 80/10/10 split. Run cells in order. After a disconnect, reconnect a GPU and run all again with the same settings and experiment name.

Training: 5 reconstruction warm-up epochs **plus 150 adversarial epochs**. Every epoch uses the entire fitting subset. No hidden subset or steps-per-epoch cap. A full run may require multiple GPU sessions. Completed stages resume or skip.

Contents

1. Environment and Drive
2. Settings
3. Validate images and prepare a persistent RGB cache
4. Fixed stratified 70/30 split and inner validation set
5. Transformations and image pipelines
6. Classifier training helpers and model A
7. Generator discriminator and perceptual network
8. Resumable SRGAN training
9. Generate the images for model B
10. Train model B and compare both classifiers
11. Export figures metrics and a reproducibility report

This is a **compact SRGAN adaptation**, not an exact reproduction of Ledig et al. Eight 32-channel residual blocks and VGG19 block3_conv4 reduce computation; reconstruction, perceptual, and adversarial losses are all retained. The report records these choices. Scores are measured during your run, never prefilled.

```
# 1. Use Colab's TensorFlow installation; do not replace its CUDA stack.
import os, sys, json, time, math, random, hashlib, shutil, zipfile, re, gc
import importlib.util, subprocess, platform, tempfile
from pathlib import Path
from datetime import datetime, timezone

for package, module in [("Pillow", "PIL"), ("scikit-learn", "sklearn"),
                        ("matplotlib", "matplotlib"), ("reportlab", "reportlab")]:
    if importlib.util.find_spec(module) is None:
        subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", package])
import numpy as np
import tensorflow as tf
from tensorflow import keras
from PIL import Image, ImageOps
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import (accuracy_score, f1_score, roc_auc_score,
    precision_score, recall_score, confusion_matrix, roc_curve, classification_report)

# Avoid a TensorFlow Grappler remapper issue with repeated discriminator calls.
tf.config.optimizer.set_experimental_options({"remapping": False})

IN_COLAB = importlib.util.find_spec("google.colab") is not None
if IN_COLAB:
    from google.colab import drive
    drive.mount("/content/drive")

gpus = tf.config.list_physical_devices("GPU")
for gpu in gpus:
    try:
        tf.config.experimental.set_memory_growth(gpu, True)
    except RuntimeError:
        pass  # Already configured by an earlier execution of this cell.
print("TensorFlow", tf.__version__, "| Keras", keras.__version__)
print("GPU:", gpus or "NONE - select a GPU runtime before the full experiment")
if int(tf.__version__.split(".")[0]) < 2 or not hasattr(keras.optimizers.Adam(), "scale_loss"):
    raise RuntimeError("This notebook needs TensorFlow with Keras 3, as in current Colab.")
```

```
Mounted at /content/drive
TensorFlow 2.21.0 | Keras 3.13.2
GPU: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

2 Settings

Edit DATA_SOURCE first. Supported layouts include `cat/` and `dog/`, `cats/` and `dogs/`, `PetImages/Cat/` and `PetImages/Dog/`, flat `cat.123.jpg` and `dog.123.jpg`, and old train/validation/test folders containing these labels. Numeric-only competition test filenames have no labels and are excluded. If the outer Kaggle download contains `train.zip`, that labeled archive is opened.

Change `EXPERIMENT_NAME` when changing the architecture, data, split, or training settings. This protects existing checkpoints from incompatible settings. `checkpoint_every=1` protects each completed epoch. Lower gan_batch to 16 or 8 if GPU memory is insufficient, then use a new experiment name.

```
DATA_SOURCE = "/content/drive/MyDrive/dogcat_dataset.zip"
EXPERIMENT_NAME = "cats_dogs_srgan_midterm_v1"
DRIVE_BASE = "/content/drive/MyDrive/SRGAN_Midterm"
LOCAL_BASE = "/content/srgan_midterm_work"

CFG = dict(seed=42, hr_size=128, lr_size=32, gan_batch=32, classifier_batch=32,
           warmup_epochs=5, gan_epochs=150, head_epochs=5, fine_epochs=3,
           g_filters=32, g_blocks=8, d_filters=32, vgg_layer="block3_conv4",
           pixel_weight=1.0, perceptual_weight=0.01, adversarial_weight=0.001,
           checkpoint_every=1, validate_every=5, session_hours=10.0,
           mixed_precision=bool(gpus), imagenet_weights="imagenet")

ROOT = Path(DRIVE_BASE) / EXPERIMENT_NAME
LOCAL = Path(LOCAL_BASE) / EXPERIMENT_NAME
for folder in [ROOT, LOCAL, ROOT / "checkpoints", ROOT / "models", ROOT / "results"]:
    folder.mkdir(parents=True, exist_ok=True)
keras.utils.set_random_seed(CFG["seed"])
POLICY = "mixed_float16" if CFG["mixed_precision"] else "float32"
keras.mixed_precision.set_global_policy(POLICY)
AUTOTUNE = tf.data.AUTOTUNE

def atomic_json(path, value):
    path = Path(path)
    tmp = path.with_name(path.name + ".tmp")
    tmp.write_text(json.dumps(value, indent=2, allow_nan=False), encoding="utf-8")
    os.replace(tmp, path)

def safe_copy(source, target):
    target = Path(target)
    target.parent.mkdir(parents=True, exist_ok=True)
    tmp = target.with_name(target.name + ".tmp")
    shutil.copyfile(source, tmp)
    os.replace(tmp, target)

def save_keras(model, target):
    target = Path(target)
    temporary_model = LOCAL / target.name
    model.save(temporary_model)
    safe_copy(temporary_model, target)

def load_json(path, default=None):
    return json.loads(Path(path).read_text()) if Path(path).exists() else default

print("Experiment:", EXPERIMENT_NAME, "| precision:", POLICY)
print("Output:", ROOT)
```

```
Experiment: cats_dogs_srgan_midterm_v1 | precision: mixed_float16
Output: /content/drive/MyDrive/SRGAN_Midterm/cats_dogs_srgan_midterm_v1
```

3 Validate images and prepare a persistent RGB cache

Images are opened with Pillow, orientation-corrected, converted to RGB, and resized once to 128 x 128. The resulting uint8 arrays avoid the JPEG decoding failures encountered in Assignment 1. Unreadable images and unknown labels are logged. Exact duplicate 128 x 128 pixel arrays are removed **before splitting**; contradictory labels cause an explicit stop instead of corrupting the experiment. The first pass takes time; subsequent sessions reuse the prepared cache.

```
IMAGE_SUFFIXES = {".jpg", ".jpeg", ".png", ".bmp", ".webp", ".tif", ".tiff"}
LABELS = {"cat": 0, "cats": 0, "dog": 1, "dogs": 1}
CLASS_NAMES = ["cat", "dog"]

def infer_label(path):
    # Prefer an explicit class folder, then Kaggle-style filenames.
    for part in reversed(path.parts[:-1]):
        if part.lower() in LABELS:
            return LABELS[part.lower()]
    match = re.match(r"^(cats?|dogs?)(?:[._\-\s]|$)", path.stem.lower())
    return LABELS[match.group(1)] if match else None

def source_signature(source):
    source = Path(source)
    if not source.exists():
        raise FileNotFoundError(
            f"Set DATA_SOURCE in Section 2 to your actual Assignment 1 ZIP/folder. Missing: {source}")
    if source.is_file():
        stat = source.stat()
        data = [str(source.resolve()), stat.st_size, stat.st_mtime_ns]
    else:
        data = [(str(p.relative_to(source)), p.stat().st_size, p.stat().st_mtime_ns)
                for p in sorted(source.rglob("*"))
                if p.is_file() and p.suffix.lower() in IMAGE_SUFFIXES]
    return hashlib.sha256(json.dumps(data).encode()).hexdigest()

def safe_extract(archive, destination):
    destination = Path(destination).resolve()
    with zipfile.ZipFile(archive) as z:
        for member in z.infolist():
            target = (destination / member.filename).resolve()
            if not target.is_relative_to(destination):
                raise ValueError("Archive contains a path outside its extraction folder.")
        z.extractall(destination)

def get_image_root(source, signature):
    source = Path(source)
    if source.is_dir():
        return source
    if source.suffix.lower() != ".zip":
        raise ValueError("DATA_SOURCE must be a ZIP or an image folder.")
    extracted = LOCAL / ("extracted_" + signature[:12])
    if not (extracted / ".complete").exists():
        extracted.mkdir(parents=True, exist_ok=True)
        local_zip = LOCAL / "dataset_source.zip"
        shutil.copyfile(source, local_zip)
        safe_extract(local_zip, extracted)
        # Kaggle's outer download may contain nested labeled train.zip.
        for inner in list(extracted.rglob("train.zip")):
            safe_extract(inner, inner.parent / "labeled_train")
        (extracted / ".complete").write_text(signature)
    return extracted

def prepare_data():
    source = Path(DATA_SOURCE)
    signature = source_signature(source)
    cache_meta = ROOT / "dataset_cache_metadata.json"
    cache_file = ROOT / "dataset_128_rgb.npz"
    cached = load_json(cache_meta, {})
    if (cache_file.exists() and cached.get("source_signature") == signature
            and cached.get("hr_size") == CFG["hr_size"]):
        local_npz = LOCAL / cache_file.name
        shutil.copyfile(cache_file, local_npz)
        with np.load(local_npz, allow_pickle=False) as z:
            images, labels = z["images"], z["labels"]
        print("Reused validated RGB cache:", len(images), "images")
        return images, labels, cached
    image_root = get_image_root(source, signature)
    files = sorted(p for p in image_root.rglob("*")
                   if p.is_file() and p.suffix.lower() in IMAGE_SUFFIXES
                   and "__MACOSX" not in p.parts and not p.name.startswith("._"))
    arrays, labels, records, rejected, duplicates = [], [], [], [], []
    seen = {}
    for number, path in enumerate(files, 1):
        label = infer_label(path)
        relative = str(path.relative_to(image_root))
        if label is None:
```

```
                rejected.append(dict(path=relative, reason="unknown label"))
                continue
            try:
                with Image.open(path) as im:
                    im = ImageOps.exif_transpose(im).convert("RGB")
                    array = np.asarray(im.resize((CFG["hr_size"], CFG["hr_size"]),
                                                 Image.Resampling.BICUBIC), dtype=np.uint8)
            except (OSError, ValueError, Image.DecompressionBombError) as error:
                rejected.append(dict(path=relative, reason=type(error).__name__))
                continue
            digest = hashlib.sha256(array.tobytes()).hexdigest()
            if digest in seen:
                old_label, old_path = seen[digest]
                if label != old_label:
                    raise ValueError(f"Identical image has conflicting labels: {relative} and {old_path}")
                duplicates.append(dict(path=relative, original=old_path))
                continue
            seen[digest] = (label, relative)
            arrays.append(array)
            labels.append(label)
            records.append(dict(index=len(labels)-1, source=relative, label=label, sha256=digest))
            if number % 1000 == 0:
                print(f"Checked {number:,}/{len(files):,} candidates", flush=True)
    if len(arrays) == 0:
        raise ValueError("No labeled images found. Use cat/dog folders or cat.0.jpg / dog.0.jpg names.")
    images, labels = np.stack(arrays), np.asarray(labels, np.int32)
    if np.bincount(labels, minlength=2).min() < 20:
        raise ValueError("Need at least 20 readable unique images of EACH class for this split.")
    fingerprint = hashlib.sha256("".join(r["sha256"] + str(r["label"]) for r in records).encode()).hexdigest()
    meta = dict(source_signature=signature, hr_size=CFG["hr_size"], count=len(images),
                class_names=CLASS_NAMES, fingerprint=fingerprint,
                rejected=len(rejected), duplicates_removed=len(duplicates))
    temporary_cache = LOCAL / "dataset_128_rgb.npz"
    np.savez_compressed(temporary_cache, images=images, labels=labels)
    safe_copy(temporary_cache, cache_file)
    atomic_json(ROOT / "image_manifest.json", records)
    atomic_json(ROOT / "rejected_images.json", rejected)
    atomic_json(ROOT / "duplicate_images.json", duplicates)
    atomic_json(cache_meta, meta)
    print("Prepared:", meta)
    return images, labels, meta

IMAGES, Y, DATA_META = prepare_data()
print("Image array:", IMAGES.shape, IMAGES.dtype, "| class counts:", np.bincount(Y))
```

```
Checked 1,000/24,930 candidates
/usr/local/lib/python3.13/dist-packages/PIL/TiffImagePlugin.py:950: UserWarning: Truncated File Read
  warnings.warn(str(msg))
Checked 2,000/24,930 candidates
Checked 3,000/24,930 candidates
Checked 4,000/24,930 candidates
Checked 5,000/24,930 candidates
Checked 6,000/24,930 candidates
Checked 7,000/24,930 candidates
Checked 8,000/24,930 candidates
Checked 9,000/24,930 candidates
Checked 10,000/24,930 candidates
Checked 11,000/24,930 candidates
Checked 12,000/24,930 candidates
Checked 13,000/24,930 candidates
Checked 14,000/24,930 candidates
Checked 15,000/24,930 candidates
Checked 16,000/24,930 candidates
Checked 17,000/24,930 candidates
Checked 18,000/24,930 candidates
Checked 19,000/24,930 candidates
Checked 20,000/24,930 candidates
Checked 21,000/24,930 candidates
Checked 22,000/24,930 candidates
Checked 23,000/24,930 candidates
Checked 24,000/24,930 candidates
Prepared: {'source_signature': '1a2b268b700204259468b60c986e4b9472956b4b999fa4a968939d102e4b3c7d', 'hr_size': 128, 'count': 2493
Image array: (24930, 128, 128, 3) uint8 | class counts: [12472 12458]
```

4 Fixed stratified 70/30 split and inner validation set

Seventy percent is the training pool and thirty percent is held out for testing. Ten percent of the training pool is validation: approximately **63% fit, 7% validation, 30% test** of the complete dataset. The generator, A, and B all fit on exactly the same fitting IDs. Neither GAN nor classifier trains on test IDs. Existing Assignment 1 split folders are treated as a source of images; the old 80/10/10 allocation is not reused. The split is saved and reused on reconnect.

```
spec_path = ROOT / "experiment_spec.json"
split_path = ROOT / "split_indices.json"
protocol = "outer70test30_inner_train_validation10_v1"

old_spec = load_json(spec_path)

if old_spec is not None:
    # Confirm this is still the same dataset.
    if old_spec.get("dataset_fingerprint") != DATA_META["fingerprint"]:
        raise ValueError(
            "The dataset differs from the saved experiment. "
            "Restore the original dataset/cache before resuming."
        )

    if old_spec.get("protocol") != protocol:
        raise ValueError("The saved experiment uses a different split protocol.")

    # Show differences, then restore the settings used for the checkpoints.
    saved_cfg = old_spec["config"]

    for key, saved_value in saved_cfg.items():
        current_value = CFG.get(key)
        if current_value != saved_value:
            print(
                f"Restoring {key}: "
                f"current={current_value!r}, saved={saved_value!r}"
            )

    if (
        saved_cfg.get("mixed_precision", False)
        and not tf.config.list_physical_devices("GPU")
    ):
        raise RuntimeError(
            "This experiment used mixed precision, but no GPU is detected. "
            "Select a GPU runtime, then rerun Sections 1–4."
        )

    CFG.update(saved_cfg)
    print("Using the original saved training settings.")

# Apply restored settings before constructing models.
POLICY = "mixed_float16" if CFG["mixed_precision"] else "float32"
keras.mixed_precision.set_global_policy(POLICY)
keras.utils.set_random_seed(CFG["seed"])

all_ids = np.arange(len(Y))
saved_split = load_json(split_path)

if old_spec is not None and saved_split is None:
    raise FileNotFoundError(
        "The saved split_indices.json is missing. "
        "Restore that file before resuming."
    )

if saved_split is not None:
    train_ids = np.asarray(saved_split["train"], dtype=np.int32)
    val_ids = np.asarray(saved_split["validation"], dtype=np.int32)
    test_ids = np.asarray(saved_split["test"], dtype=np.int32)

    print("Reused the saved training, validation, and test IDs.")

else:
    pool_ids, test_ids = train_test_split(
        all_ids,
        test_size=0.30,
        random_state=CFG["seed"],
        stratify=Y
    )

    train_ids, val_ids = train_test_split(
```

```
            pool_ids,
            test_size=0.10,
            random_state=CFG["seed"],
            stratify=Y[pool_ids]
        )

        train_ids, val_ids, test_ids = [
            np.sort(ids).astype(np.int32)
            for ids in (train_ids, val_ids, test_ids)
        ]

    # Verify every image belongs to exactly one subset.
    joined_ids = np.concatenate([train_ids, val_ids, test_ids])

    if not np.array_equal(np.sort(joined_ids), all_ids):
        raise ValueError(
            "Saved split IDs do not match this dataset "
            "or contain overlapping/missing images."
        )

    SPLIT = {
        "train": train_ids.tolist(),
        "validation": val_ids.tolist(),
        "test": test_ids.tolist()
    }

    fixed_cfg = {
        key: value
        for key, value in CFG.items()
        if key not in {"session_hours", "checkpoint_every", "validate_every"}
    }

    RUN_SPEC = dict(
        config=fixed_cfg,
        dataset_fingerprint=DATA_META["fingerprint"],
        protocol=protocol
    )

    # Existing metadata and checkpoints are left intact.
    if saved_split is None:
        atomic_json(split_path, SPLIT)

    if old_spec is None:
        atomic_json(spec_path, RUN_SPEC)

    for name, ids in [
        ("Fitting", train_ids),
        ("Validation", val_ids),
        ("Test", test_ids)
    ]:
        print(
            f"{name:12s} {len(ids):6,d} images  "
            f"cat/dog: {np.bincount(Y[ids], minlength=2)}"
        )

    print("Positive class = dog (1); negative class = cat (0)")
    print("Precision:", POLICY)
    print("Existing checkpoints were preserved.")
```

```
Using the original saved training settings.
Reused the saved training, validation, and test IDs.
Fitting      15,705 images  cat/dog: [7857 7848]
Validation    1,746 images  cat/dog: [873 873]
Test          7,479 images  cat/dog: [3742 3737]
Positive class = dog (1); negative class = cat (0)
Precision: mixed_float16
Existing checkpoints were preserved.
```

5 Transformations normalization and image pipelines

Stored images use uint8 [0,255]. GAN and classifier input arrays use float32 [0,1]. MobileNetV2 preprocessing is inside the classifier: x -> 2x - 1. VGG19 receives its own ImageNet BGR mean-subtracted preprocessing.

GAN pairs share a random horizontal flip before the HR image is downsampled; this preserves alignment. Classifiers use horizontal flips, small rotations, zoom, and contrast changes during training only. Validation/test are deterministic.

```
with tf.device("/CPU:0"):
    HR_BANK = tf.convert_to_tensor(IMAGES, dtype=tf.uint8)
    LABEL_BANK = tf.convert_to_tensor(Y, dtype=tf.int32)

def make_image_ds(ids, bank=None, batch=None, shuffle=False, epoch_seed=None):
    bank = HR_BANK if bank is None else bank
    batch = CFG["classifier_batch"] if batch is None else batch
    ds = tf.data.Dataset.from_tensor_slices(np.asarray(ids, np.int32))
    if shuffle:
        ds = ds.shuffle(len(ids), seed=epoch_seed or CFG["seed"], reshuffle_each_iteration=False)
    def fetch(i):
        with tf.device("/CPU:0"):
            image = tf.gather(bank, i)
            label = tf.gather(LABEL_BANK, i)
        return tf.cast(image, tf.float32) / 255.0, tf.cast(label, tf.float32)
    return ds.map(fetch, num_parallel_calls=AUTOTUNE).batch(batch).prefetch(AUTOTUNE)

def make_gan_ds(ids, training=False, epoch=0):
    ds = make_image_ds(ids, batch=CFG["gan_batch"], shuffle=training,
                       epoch_seed=CFG["seed"] + epoch)
    def pair(hr, labels):
        if training:
            hr = tf.image.random_flip_left_right(hr, seed=CFG["seed"])
        lr = tf.image.resize(hr, (CFG["lr_size"], CFG["lr_size"]), method="bicubic", antialias=True)
        return tf.clip_by_value(lr, 0.0, 1.0), hr
    return ds.map(pair, num_parallel_calls=AUTOTUNE).prefetch(AUTOTUNE)

def build_augmentation():
    return keras.Sequential([
        keras.layers.RandomFlip("horizontal", seed=CFG["seed"]),
        keras.layers.RandomRotation(0.04, fill_mode="reflect", seed=CFG["seed"]+1),
        keras.layers.RandomZoom(0.10, fill_mode="reflect", seed=CFG["seed"]+2),
        keras.layers.RandomContrast(0.10, seed=CFG["seed"]+3),
    ], name="training_augmentation")

def save_figure(name):
    plt.savefig(ROOT / "results" / name, dpi=150, bbox_inches="tight")
    plt.show()
    plt.close()

sample = IMAGES[train_ids[0]].astype(np.float32) / 255.0
augmented = tf.cast(tf.clip_by_value(build_augmentation()(
    np.repeat(sample[None], 6, axis=0), training=True),0,1),tf.float32).numpy()
fig, axes = plt.subplots(1, 6, figsize=(13, 2.6))
for i, ax in enumerate(axes):
    ax.imshow(augmented[i]); ax.set_title(f"Transform {i+1}"); ax.axis("off")
save_figure("transformed_samples.png")

lr_sample = tf.clip_by_value(tf.image.resize(sample, (32,32), method="bicubic", antialias=True),0,1)
bic_sample = tf.clip_by_value(tf.image.resize(lr_sample, (128,128), method="bicubic"),0,1)
fig, axes = plt.subplots(1, 3, figsize=(9,3))
for ax, image, title in zip(axes, [sample,lr_sample,bic_sample],
                            ["Original target 128 x 128", "Downscaled 32 x 32", "Bicubic 128 x 128"]):
    ax.imshow(image, interpolation="nearest"); ax.set_title(title); ax.axis("off")
save_figure("scaled_samples.png")
print("GAN input range [0,1]; MobileNetV2 range [-1,1].")
```




```
GAN input range [0,1]; MobileNetV2 range [-1,1].
```

6 Transfer learning helpers and classifier A

Both models start from the **same saved initialization**, use MobileNetV2 at 128 x 128, and train for 5 frozen-backbone epochs plus 3 fine-tuning epochs. During fine-tuning the final 30 backbone layers can learn, except batch normalization, which remains frozen. Learning rates are 1e-3 then 1e-5. The checkpoint with the lowest validation loss across both stages is exported.

Each completed classifier epoch saves model weights, optimizer state, epoch, and best validation loss. Rerunning this section restores the latest completed epoch. Batch order is seeded by epoch; reconnecting is not promised to be bitwise identical to an uninterrupted run because random augmentation state can differ.

```
def build_classifier():
    inp = keras.Input((128,128,3), name="rgb_zero_to_one")
    x = build_augmentation()(inp)
    x = keras.layers.Rescaling(2.0, offset=-1.0, name="mobilenet_preprocess")(x)
    base = keras.applications.MobileNetV2(include_top=False,
        weights=CFG["imagenet_weights"], input_shape=(128,128,3))
    base.trainable = False
    x = base(x, training=False)
    x = keras.layers.GlobalAveragePooling2D()(x)
    x = keras.layers.Dropout(0.30, seed=CFG["seed"]+4)(x)
    out = keras.layers.Dense(1, activation="sigmoid", dtype="float32", name="p_dog")(x)
    return keras.Model(inp, out, name="cats_dogs_mobilenetv2")

initial_path = ROOT / "models" / "classifier_initial.keras"
if not initial_path.exists():
    keras.utils.set_random_seed(CFG["seed"])
    initial_classifier = build_classifier()
    save_keras(initial_classifier, initial_path)
    del initial_classifier
    gc.collect()

def make_optimizer(lr):
    optimizer = keras.optimizers.Adam(learning_rate=lr)
    return keras.mixed_precision.LossScaleOptimizer(optimizer) if CFG["mixed_precision"] else optimizer

def classifier_stage(name, stage, model, bank, epochs, lr):
    folder = ROOT / "checkpoints" / name / stage
    folder.mkdir(parents=True, exist_ok=True)
    model.compile(optimizer=make_optimizer(lr), loss="binary_crossentropy",
                  metrics=[keras.metrics.BinaryAccuracy(name="accuracy"),
                           keras.metrics.AUC(name="auc")], jit_compile=False)
    model.optimizer.build(model.trainable_variables)
    completed = tf.Variable(0, dtype=tf.int64, trainable=False)
    best_loss = tf.Variable(float("inf"), dtype=tf.float32, trainable=False)
    checkpoint = tf.train.Checkpoint(model=model, optimizer=model.optimizer,
                                     epoch=completed, best_loss=best_loss)
    manager = tf.train.CheckpointManager(checkpoint, str(folder / "latest"), max_to_keep=2)
    best_manager = tf.train.CheckpointManager(checkpoint, str(folder / "best"), max_to_keep=1)
```

```
    if manager.latest_checkpoint:
        checkpoint.restore(manager.latest_checkpoint).assert_existing_objects_matched()
        print(name, stage, "resuming after epoch", int(completed.numpy()))
    history_path = folder / "history.json"
    history = load_json(history_path, [])
    # Trim rows from work beyond the most recent completed checkpoint, if any.
    history = [r for r in history if r["epoch"] <= int(completed.numpy())]
    for epoch in range(int(completed.numpy()), epochs):
        train_ds = make_image_ds(train_ids, bank=bank, shuffle=True,
            epoch_seed=CFG["seed"] + epoch + (100 if stage == "fine" else 0))
        val_ds = make_image_ds(val_ids, bank=bank)
        start = time.time()
        result = model.fit(train_ds, validation_data=val_ds, epochs=1, verbose=2, shuffle=False).history
        loss = float(result["val_loss"][-1])
        if not math.isfinite(loss):
            raise FloatingPointError("Classifier validation loss is nonfinite; checkpoint was not advanced.")
        improved = loss < float(best_loss.numpy())
        if improved:
            best_loss.assign(loss)
        completed.assign(epoch+1)
        manager.save(checkpoint_number=epoch+1)
        if improved:
            best_manager.save(checkpoint_number=epoch+1)
        row = dict(epoch=epoch+1, seconds=time.time()-start,
                   **{k: float(v[-1]) for k,v in result.items()})
        history.append(row)
        atomic_json(history_path, history)
        print(f"{name} {stage}: saved completed epoch {epoch+1}/{epochs}", flush=True)
    if best_manager.latest_checkpoint is None:
        raise RuntimeError("No valid classifier checkpoint exists for this stage.")
    checkpoint.restore(best_manager.latest_checkpoint).assert_existing_objects_matched()
    return model, float(best_loss.numpy())

def train_classifier(name, bank):
    output = ROOT / "models" / f"classifier_{name}.keras"
    if output.exists():
        print(f"Classifier {name} already complete; loading exported best model.")
        return keras.models.load_model(output, compile=False)
    model = keras.models.load_model(initial_path, compile=False)
    model, head_loss = classifier_stage(name,"head",model,bank,CFG["head_epochs"],1e-3)
    head_export = ROOT / "models" / f"classifier_{name}_head.keras"
    save_keras(model, head_export)
    base = next(layer for layer in model.layers if isinstance(layer, keras.Model)
                and "mobilenet" in layer.name.lower())
    base.trainable = True
    for layer in base.layers[:-30]:
        layer.trainable = False
    for layer in base.layers[-30:]:
        if isinstance(layer, keras.layers.BatchNormalization):
            layer.trainable = False
    model, fine_loss = classifier_stage(name,"fine",model,bank,CFG["fine_epochs"],1e-5)
    selected = "fine" if fine_loss < head_loss else "head"
    if selected == "head":
        model = keras.models.load_model(head_export, compile=False)
    save_keras(model, output)
    atomic_json(ROOT / "models" / f"classifier_{name}_selection.json",
                dict(stage=selected, head_val_loss=head_loss, fine_val_loss=fine_loss))
    return model

MODEL_A = train_classifier("A", HR_BANK)
print("Classifier A complete. Test images remain reserved for final evaluation.")
```

```
Classifier A already complete; loading exported best model.
Classifier A complete. Test images remain reserved for final evaluation.
```

7 Generator discriminator and perceptual network

The generator operates mostly at 32 x 32, then uses two pixel-shuffle blocks to enlarge each spatial dimension by 2 twice. Its sigmoid output is [0,1]. The discriminator outputs a logit; binary cross-entropy uses `from_logits=True`. VGG19 is frozen and supplies feature content loss. It does not classify cats/dogs.

Generator objective during adversarial training: `L_G = 1.0 * mean(abs(SR-HR)) + 0.01 * mean((VGG(SR)/12.75 - VGG(HR)/12.75)^2) + 0.001 * BCE(real, D(SR))`. Discriminator objective: average of real and fake BCE, with real labels 0.9 and fake labels 0.0. Generator real targets remain 1.0.

```
@keras.utils.register_keras_serializable(package="SRGANMidterm")
class PixelShuffle(keras.layers.Layer):
    def __init__(self, scale=2, **kwargs):
        super().__init__(**kwargs)
        self.scale = scale
    def call(self, x):
        return tf.nn.depth_to_space(x, self.scale)
    def compute_output_shape(self, shape):
        return (shape[0], None if shape[1] is None else shape[1]*self.scale,
                None if shape[2] is None else shape[2]*self.scale,
                shape[3]//(self.scale**2))
    def get_config(self):
        return dict(super().get_config(), scale=self.scale)

def build_generator():
    f = CFG["g_filters"]
    inp = keras.Input((32,32,3))
    x = keras.layers.Conv2D(f,9,padding="same")(inp)
    x = keras.layers.PReLU(shared_axes=[1,2])(x)
    long_skip = x
    for _ in range(CFG["g_blocks"]):
        residual = x
        x = keras.layers.Conv2D(f,3,padding="same",use_bias=False)(x)
        x = keras.layers.BatchNormalization(momentum=0.8)(x)
        x = keras.layers.PReLU(shared_axes=[1,2])(x)
        x = keras.layers.Conv2D(f,3,padding="same",use_bias=False)(x)
        x = keras.layers.BatchNormalization(momentum=0.8)(x)
        x = keras.layers.Add()([residual,x])
    x = keras.layers.Conv2D(f,3,padding="same",use_bias=False)(x)
    x = keras.layers.BatchNormalization(momentum=0.8)(x)
    x = keras.layers.Add()([long_skip,x])
    for _ in range(2):
        x = keras.layers.Conv2D(f*4,3,padding="same")(x)
        x = PixelShuffle(2)(x)
        x = keras.layers.PReLU(shared_axes=[1,2])(x)
    out = keras.layers.Conv2D(3,9,padding="same",activation="sigmoid",dtype="float32")(x)
    return keras.Model(inp,out,name="srgan_generator")

def build_discriminator():
    inp = keras.Input((128,128,3))
    x = inp
    for i, (multiplier,stride) in enumerate([(1,1),(1,2),(2,1),(2,2),(4,1),(4,2),(8,1),(8,2)]):
        x = keras.layers.Conv2D(CFG["d_filters"]*multiplier,3,strides=stride,padding="same")(x)
        if i:
            x = keras.layers.BatchNormalization(momentum=0.8)(x)
        x = keras.layers.LeakyReLU(negative_slope=0.2)(x)
    x = keras.layers.GlobalAveragePooling2D()(x)
    x = keras.layers.Dense(256)(x)
    x = keras.layers.LeakyReLU(negative_slope=0.2)(x)
    return keras.Model(inp,keras.layers.Dense(1,dtype="float32")(x),name="srgan_discriminator")

G, D = build_generator(), build_discriminator()
vgg_base = keras.applications.VGG19(include_top=False, weights=CFG["imagenet_weights"],
                                    input_shape=(128,128,3))
VGG = keras.Model(vgg_base.input,vgg_base.get_layer(CFG["vgg_layer"]).output)
VGG.trainable = False
del vgg_base
print("Generator:", G.count_params(), "parameters", G.input_shape,"->",G.output_shape)
print("Discriminator:", D.count_params(), "parameters | VGG feature:",CFG["vgg_layer"])
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg19/vgg19_weights_tf_dim_ordering_tf_kernel
80134624/80134624 ━━━━━━━━━━━━━━━━━━━━ 4s 0us/step
Generator: 248771 parameters (None, 32, 32, 3) -> (None, 128, 128, 3)
Discriminator: 1242017 parameters | VGG feature: block3_conv4
```

8 Resumable SRGAN training

The long-running cell is below. Warm-up does not count toward the required 150 adversarial epochs. An epoch is one complete pass through the fitting set. No GAN early stopping is used. Validation is measured every 5 epochs and at the last epoch. Both networks, both optimizers, mixed-precision loss scales, and both epoch counters are checkpointed to Drive.

A graceful session budget is checked at epoch boundaries. If it is reached, later cells skip unfinished stages and you can reconnect and rerun all. A sudden disconnect can lose work since the latest saved epoch; rerun all with the same experiment. With the default save interval, at most the active epoch is lost. The final generator after all 150 epochs supplies model B.

```
G_OPT, D_OPT = make_optimizer(1e-4), make_optimizer(1e-4)
G_OPT.build(G.trainable_variables)
D_OPT.build(D.trainable_variables)
warmup_done = tf.Variable(0, dtype=tf.int64, trainable=False)
gan_done = tf.Variable(0, dtype=tf.int64, trainable=False)
GAN_CKPT = tf.train.Checkpoint(generator=G, discriminator=D, g_optimizer=G_OPT,
    d_optimizer=D_OPT, warmup_epoch=warmup_done, adversarial_epoch=gan_done)
GAN_MANAGER = tf.train.CheckpointManager(GAN_CKPT, str(ROOT / "checkpoints" / "srgan"), max_to_keep=3)
if GAN_MANAGER.latest_checkpoint:
    GAN_CKPT.restore(GAN_MANAGER.latest_checkpoint).assert_existing_objects_matched()
    print("Restored warm-up:",int(warmup_done.numpy()),"| adversarial:",int(gan_done.numpy()))

BCE = keras.losses.BinaryCrossentropy(from_logits=True)
def scaled_loss(optimizer, loss):
    return optimizer.scale_loss(loss) if CFG["mixed_precision"] else loss

@tf.function(reduce_retracing=True, jit_compile=False)
def warmup_step(lr, hr):
    with tf.GradientTape() as tape:
        sr = G(lr, training=True)
        loss = tf.reduce_mean(tf.abs(sr - hr))
        tf.debugging.check_numerics(loss,"Nonfinite reconstruction loss")
        scaled = scaled_loss(G_OPT,loss)
    gradients = tape.gradient(scaled,G.trainable_variables)
    G_OPT.apply_gradients(zip(gradients,G.trainable_variables))
    return loss

@tf.function(reduce_retracing=True, jit_compile=False)
def adversarial_step(lr, hr):
    real_input = keras.applications.vgg19.preprocess_input(tf.cast(hr,tf.float32)*255.0)
    real_features = tf.stop_gradient(tf.cast(VGG(real_input,training=False),tf.float32)/12.75)
    with tf.GradientTape() as g_tape, tf.GradientTape() as d_tape:
        sr = G(lr, training=True)
        real_logits = D(hr, training=True)
        fake_logits = D(tf.stop_gradient(sr), training=True)
        # Inference mode prevents G's objective from updating D's batch-norm state.
        generator_logits = D(sr, training=False)
        generated_input = keras.applications.vgg19.preprocess_input(sr*255.0)
        generated_features = tf.cast(VGG(generated_input,training=False),tf.float32)/12.75
        pixel = tf.reduce_mean(tf.abs(sr-hr))
        content = tf.reduce_mean(tf.square(generated_features-real_features))
        adversarial = BCE(tf.ones_like(generator_logits),generator_logits)
        g_loss = (CFG["pixel_weight"]*pixel + CFG["perceptual_weight"]*content
                  + CFG["adversarial_weight"]*adversarial)
        d_loss = 0.5*(BCE(tf.ones_like(real_logits)*0.9,real_logits)
                      + BCE(tf.zeros_like(fake_logits),fake_logits))
        tf.debugging.check_numerics(g_loss,"Nonfinite generator loss")
        tf.debugging.check_numerics(d_loss,"Nonfinite discriminator loss")
        g_scaled, d_scaled = scaled_loss(G_OPT,g_loss), scaled_loss(D_OPT,d_loss)
    g_grads = g_tape.gradient(g_scaled,G.trainable_variables)
    d_grads = d_tape.gradient(d_scaled,D.trainable_variables)
    G_OPT.apply_gradients(zip(g_grads,G.trainable_variables))
    D_OPT.apply_gradients(zip(d_grads,D.trainable_variables))
    return tf.stack([g_loss,d_loss,pixel,content,adversarial])

def image_quality(ids):
    totals = np.zeros(4,dtype=np.float64)
    count = 0
    for lr, hr in make_gan_ds(ids):
        sr = tf.clip_by_value(G(lr,training=False),0.0,1.0)
        bicubic = tf.clip_by_value(tf.image.resize(lr,(128,128),method="bicubic"),0.0,1.0)
        # Exact constant-image matches have infinite PSNR; cap at 100 dB for JSON.
```

```
        values = [tf.minimum(tf.image.psnr(sr,hr,1.0),100.0),tf.image.ssim(sr,hr,1.0),
                  tf.minimum(tf.image.psnr(bicubic,hr,1.0),100.0),tf.image.ssim(bicubic,hr,1.0)]
        totals += np.array([float(tf.reduce_sum(v)) for v in values])
        count += int(tf.shape(hr)[0])
    return dict(zip(["sr_psnr","sr_ssim","bicubic_psnr","bicubic_ssim"],(totals/count).tolist()))

def preview_sr(ids, filename):
    ids = np.asarray(ids,np.int32)
    ids = np.asarray([i for label in [0,1] for i in ids[Y[ids]==label][:2]],np.int32)
    hr = IMAGES[ids].astype(np.float32)/255.0
    lr = tf.clip_by_value(tf.image.resize(hr,(32,32),method="bicubic",antialias=True),0,1)
    bic = tf.clip_by_value(tf.image.resize(lr,(128,128),method="bicubic"),0,1)
    sr = G(lr,training=False).numpy()
    fig,axes = plt.subplots(len(ids),4,figsize=(10,2.4*len(ids)),squeeze=False)
    for row in range(len(ids)):
        for col,(image,title) in enumerate([(lr[row],"LR 32"),(bic[row],"Bicubic 128"),
                                            (sr[row],"SRGAN 128"),(hr[row],"Real HR 128")]):
            axes[row,col].imshow(np.clip(image,0,1),interpolation="nearest")
            axes[row,col].set_title(title); axes[row,col].axis("off")
    save_figure(filename)

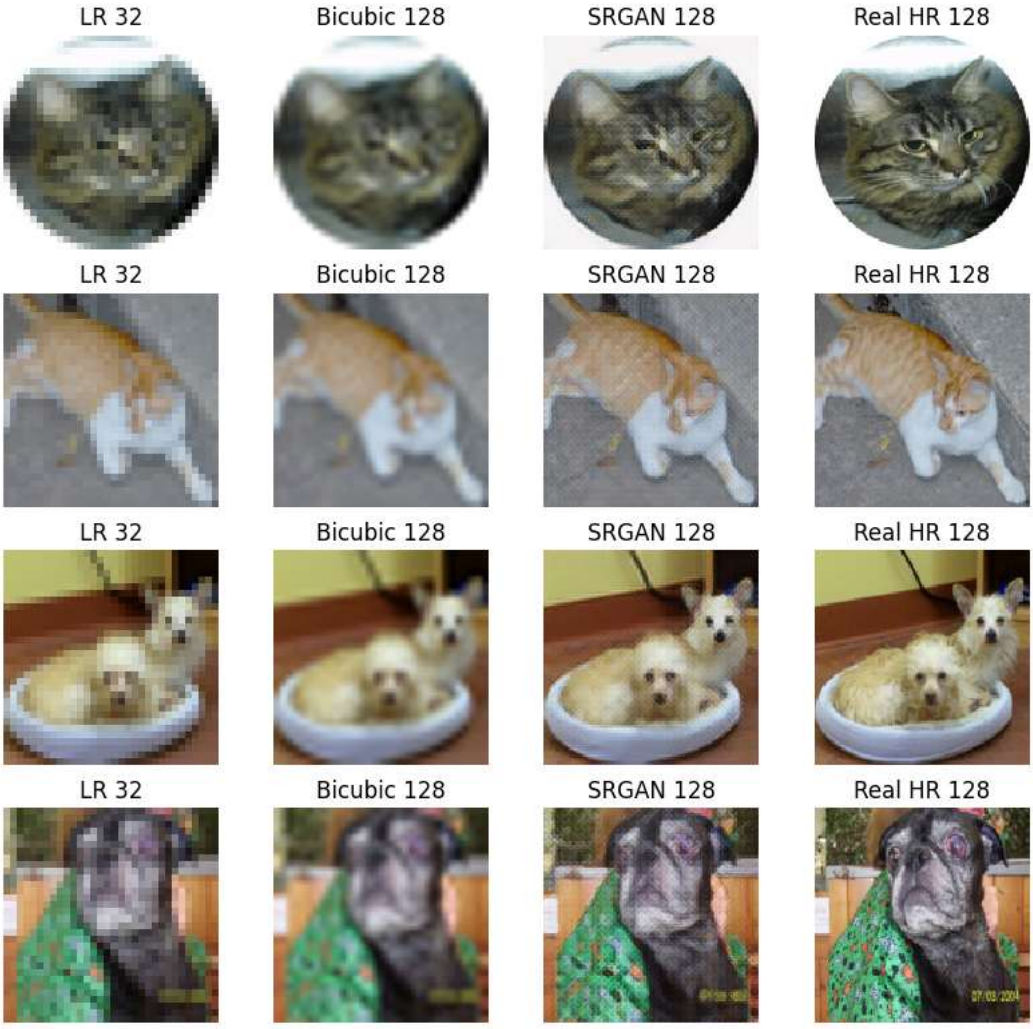
def train_srgan():
    start_session = time.monotonic()
    history_path = ROOT / "results" / "srgan_history.json"
    history = load_json(history_path, [])
    history = [row for row in history if row["epoch"] <= int(
        warmup_done.numpy() if row["phase"] == "warmup" else gan_done.numpy())]
    for phase,target,counter in [("warmup",CFG["warmup_epochs"],warmup_done),
                                 ("adversarial",CFG["gan_epochs"],gan_done)]:
        for epoch in range(int(counter.numpy()),target):
            start = time.monotonic()
            seen = 0
            for lr,hr in make_gan_ds(train_ids,training=True,epoch=epoch):
                loss = warmup_step(lr,hr) if phase == "warmup" else adversarial_step(lr,hr)
                # Retain all G/D loss components with sample-weighted accumulation.
                if seen == 0:
                    sums = np.zeros(1 if phase == "warmup" else 5,dtype=np.float64)
                n = int(tf.shape(hr)[0])
                sums += np.asarray(loss.numpy()).reshape(-1)*n
                seen += n
            assert seen == len(train_ids), "An epoch did not visit every fitting image."
            counter.assign(epoch+1)
            row = dict(phase=phase,epoch=epoch+1,images_seen=seen,
                       seconds=time.monotonic()-start,losses=(sums/seen).tolist())
            if phase == "adversarial" and ((epoch+1)%CFG["validate_every"]==0
                    or epoch==0 or epoch+1==target):
                row.update(image_quality(val_ids))
            history.append(row)
            need_save = ((epoch+1)%CFG["checkpoint_every"]==0 or epoch+1==target)
            if need_save:
                GAN_MANAGER.save()
                atomic_json(history_path,history)
            print(f"{phase}: {epoch+1}/{target}, {seen:,} images, {row['seconds']:.1f}s, "
                  f"losses={np.round(row['losses'],5)}" + (" | saved" if need_save else ""),flush=True)
            if phase == "adversarial":
                remaining_hours = (target-epoch-1)*row["seconds"]/3600
                print(f"Rough remaining adversarial time at this epoch's speed: {remaining_hours:.2f} GPU hours",flush=True)
            if phase == "adversarial" and ((epoch+1)%25==0 or epoch+1==target):
                preview_sr(val_ids,f"validation_epoch_{epoch+1:03d}.png")
            if (time.monotonic()-start_session)/3600 >= CFG["session_hours"]:
                GAN_MANAGER.save()
                atomic_json(history_path,history)
                print("Session budget reached. Reconnect a GPU and run all again to continue.")
                return
    GAN_MANAGER.save()
    atomic_json(history_path,history)

if gpus or not IN_COLAB:
    train_srgan()
else:
    print("Full SRGAN training skipped: switch this Colab runtime to GPU and rerun all.")

GAN_COMPLETE = (int(warmup_done.numpy()) >= CFG["warmup_epochs"]
                and int(gan_done.numpy()) >= CFG["gan_epochs"])
print("Warm-up epochs:",int(warmup_done.numpy()),"| adversarial epochs:",int(gan_done.numpy()))
if GAN_COMPLETE:
    save_keras(G,ROOT/"models"/"generator_final.keras")
```

```
    save_keras(D,ROOT/"models"/"discriminator_final.keras")
    preview_sr(val_ids,"validation_final.png")
else:
    print("SRGAN is unfinished. Model B, final testing, and submission export will wait.")
```

```
Restored warm-up: 5 | adversarial: 150
Warm-up epochs: 5 | adversarial epochs: 150
```



9 Generate the images for model B

G is now frozen for image generation. Each generated image keeps the class label and split ID of its LR source. Model B uses **only generated fitting images**; its validation inputs are generated validation images. Test reconstructions are created for the additional input-condition comparison, never used for fitting. Generated RGB arrays are saved once and reused; G is not rerun inside every B training epoch. This saves substantial GPU time.

```
SR_BANK = None
SR_IMAGES = None
if GAN_COMPLETE:
    # The weight hash is stable even if .keras ZIP timestamps change after an export.
    generator_weight_hash = hashlib.sha256(b"".join(w.tobytes() for w in G.get_weights())).hexdigest()
    generation_spec = dict(dataset=DATA_META["fingerprint"],generator_weights=generator_weight_hash,
                           adversarial_epochs=int(gan_done.numpy()))
    sr_cache = ROOT / "sr_generated_128_rgb.npz"
    if sr_cache.exists() and load_json(ROOT/"sr_cache_metadata.json") == generation_spec:
        shutil.copyfile(sr_cache,LOCAL/sr_cache.name)
        with np.load(LOCAL/sr_cache.name,allow_pickle=False) as z:
            SR_IMAGES=z["images"]
        print("Reused generated image cache.")
    else:
        SR_IMAGES=np.empty_like(IMAGES)
        offset=0
        # all_ids is sorted and this dataset never shuffles, so IDs stay aligned.
```

```
            for hr,labels in make_image_ds(all_ids,batch=CFG["gan_batch"]):
                lr=tf.clip_by_value(tf.image.resize(hr,(32,32),method="bicubic",antialias=True),0,1)
                generated=G(lr,training=False).numpy()
                n=len(generated)
                SR_IMAGES[offset:offset+n]=np.rint(np.clip(generated,0,1)*255).astype(np.uint8)
                offset+=n
            assert offset == len(IMAGES)
            np.savez_compressed(LOCAL/sr_cache.name,images=SR_IMAGES)
            safe_copy(LOCAL/sr_cache.name,sr_cache)
            atomic_json(ROOT/"sr_cache_metadata.json",generation_spec)
        assert SR_IMAGES.shape == IMAGES.shape
        with tf.device("/CPU:0"):
            SR_BANK=tf.convert_to_tensor(SR_IMAGES,dtype=tf.uint8)
        print("Generated images:",SR_IMAGES.shape,"| labels retained:",np.bincount(Y))
    else:
        print("Waiting for all adversarial epochs. No partial SRGAN images will train model B.")
```

```
Reused generated image cache.
Generated images: (24930, 128, 128, 3) | labels retained: [12472 12458]
```

10 Train classifier B and compare both classifiers

The **primary comparison** evaluates A and B on exactly the same real 128 x 128 test images. This isolates the effect of changing training-image quality. A secondary table evaluates **both** classifiers on the same bicubic images and the same SRGAN images from the test IDs. This avoids comparing scores obtained under different input conditions as though they were directly equivalent.

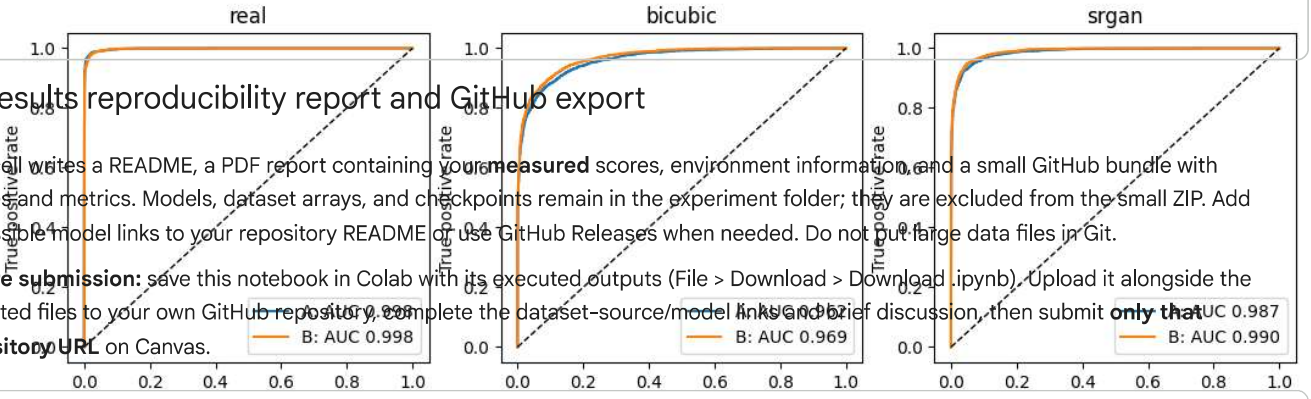
Accuracy, binary F1, precision, and recall use a fixed probability threshold 0.5. ROC AUC uses **probabilities**, not thresholded labels. Dog is the positive class. Reconstruction PSNR/SSIM compare SR and bicubic against the real targets. Better visual appearance, higher PSNR, and higher classification accuracy need not agree.

```
MODEL_B = train_classifier("B",SR_BANK) if GAN_COMPLETE else None
METRICS = []
TEST_QUALITY = None
if MODEL_B is not None:
    y_true=Y[test_ids]
    def evaluation_ds(condition):
        if condition == "real":
            return make_image_ds(test_ids)
        if condition == "srgan":
            return make_image_ds(test_ids,bank=SR_BANK)
        def bicubic(x,y):
            lr=tf.clip_by_value(tf.image.resize(x,(32,32),method="bicubic",antialias=True),0,1)
            hr=tf.clip_by_value(tf.image.resize(lr,(128,128),method="bicubic"),0,1)
            return hr,y
        return make_image_ds(test_ids).map(bicubic,num_parallel_calls=AUTOTUNE).prefetch(AUTOTUNE)
    predictions={}
    for condition in ["real","bicubic","srgan"]:
        ds=evaluation_ds(condition)
        for name,model in [("A",MODEL_A),("B",MODEL_B)]:
            probabilities=model.predict(ds,verbose=0).reshape(-1)
            assert len(probabilities)==len(y_true) and np.isfinite(probabilities).all()
            predicted=(probabilities>=0.5).astype(np.int32)
            row=dict(model=name,test_input=condition,n=len(y_true),
                accuracy=float(accuracy_score(y_true,predicted)),
                f1=float(f1_score(y_true,predicted,zero_division=0)),
                auc=float(roc_auc_score(y_true,probabilities)),
                precision=float(precision_score(y_true,predicted,zero_division=0)),
                recall=float(recall_score(y_true,predicted,zero_division=0)))
            METRICS.append(row)
            predictions[(name,condition)]=probabilities
            np.savetxt(ROOT/"results"/f"predictions_{name}_{condition}.csv",
                np.column_stack([test_ids,y_true,probabilities,predicted]),delimiter=",",
                header="image_id,true_label,p_dog,predicted_label",comments="")
            (ROOT/"results"/f"classification_{name}_{condition}.txt").write_text(
                classification_report(y_true,predicted,target_names=CLASS_NAMES,zero_division=0))
    atomic_json(ROOT/"results"/"classifier_metrics.json",METRICS)
    import csv
    with (ROOT/"results"/"classifier_metrics.csv").open("w",newline="") as f:
        writer=csv.DictWriter(f,fieldnames=list(METRICS[0]))
        writer.writeheader();writer.writerows(METRICS)
    print("PRIMARY COMPARISON - identical REAL test images")
    print("Model   Accuracy      F1        AUC")
    for row in METRICS:
        if row["test_input"]=="real":
```

```
            print(f"{row['model']:5s}   {row['accuracy']:.4f}     {row['f1']:.4f}    {row['auc']:.4f}")
    print("\nAll input conditions:")
    for row in METRICS: print(row)
    fig,axes=plt.subplots(1,3,figsize=(13,3.5))
    for ax,condition in zip(axes,["real","bicubic","srgan"]):
        for name in ["A","B"]:
            fpr,tpr,_=roc_curve(y_true,predictions[(name,condition)])
            auc=roc_auc_score(y_true,predictions[(name,condition)])
            ax.plot(fpr,tpr,label=f"{name}: AUC {auc:.3f}")
        ax.plot([0,1],[0,1],"k--",linewidth=1);ax.set_title(condition)
        ax.set_xlabel("False positive rate");ax.set_ylabel("True positive rate");ax.legend()
    save_figure("roc_curves.png")
    fig,axes=plt.subplots(1,2,figsize=(8,3.5))
    for ax,name in zip(axes,["A","B"]):
        cm=confusion_matrix(y_true,predictions[(name,"real")]>=0.5,labels=[0,1])
        ax.imshow(cm,cmap="Blues")
        for (i,j),value in np.ndenumerate(cm):ax.text(j,i,str(value),ha="center",va="center")
        ax.set(xticks=[0,1],yticks=[0,1],xticklabels=CLASS_NAMES,yticklabels=CLASS_NAMES,
               xlabel="Predicted",ylabel="Actual",title=f"Model {name} real test")
    save_figure("confusion_matrices.png")
    TEST_QUALITY=image_quality(test_ids)
    atomic_json(ROOT/"results"/"test_image_quality.json",TEST_QUALITY)
    print("Test reconstruction metrics:",TEST_QUALITY)
    preview_sr(test_ids,"test_sr_examples.png")
else:
    print("Comparison waits until SRGAN and classifier B finish.")
```

```
Classifier B already complete; loading exported best model.
PRIMARY COMPARISON - identical REAL test images
Model   Accuracy      F1        AUC
A       0.9806     0.9806    0.9980
B       0.9713     0.9718    0.9978

All input conditions:
{'model': 'A', 'test_input': 'real', 'n': 7479, 'accuracy': 0.980612381334403, 'f1': 0.9805916209342792, 'auc': 0.99799293528093
{'model': 'B', 'test_input': 'real', 'n': 7479, 'accuracy': 0.9712528412889424, 'f1': 0.9717736641722463, 'auc': 0.9977540168826
{'model': 'A', 'test_input': 'bicubic', 'n': 7479, 'accuracy': 0.879261933413558, 'f1': 0.8685780817930432, 'auc': 0.96223680538
{'model': 'B', 'test_input': 'bicubic', 'n': 7479, 'accuracy': 0.8985158443642198, 'f1': 0.9001972386587771, 'auc': 0.9692253651
{'model': 'A', 'test_input': 'srgan', 'n': 7479, 'accuracy': 0.9405000668538575, 'f1': 0.9400511922403341, 'auc': 0.986828523810
{'model': 'B', 'test_input': 'srgan', 'n': 7479, 'accuracy': 0.9434416365824309, 'f1': 0.9443933219403181, 'auc': 0.989524811972
```



11 Results reproducibility report and GitHub export

This cell writes a README, a PDF report containing your **measured** scores, environment information, and a small GitHub bundle with figures and metrics. Models, dataset arrays, and checkpoints remain in the experiment folder; they are excluded from the small ZIP. Add accessible model links to your repository README or use GitHub Releases when needed. Do not put large data files in Git.

Before submission: save this notebook in Colab with its executed outputs (File > Download > Download .ipynb). Upload it alongside the exported files to your own GitHub repository, complete the dataset-source/model links and brief discussion, then submit **only that repository URL** on Canvas.

```
def markdown_table(rows):
    lines=["| Model | Test input | Accuracy | F1 | ROC AUC | Precision | Recall |",
           "|---|---|---:|---:|---:|---:|---:|"]
    for r in rows:
        lines.append(f"| {r['model']} | {r['test_input']} | {r['accuracy']:.4f} | {r['f1']:.4f} | "
                     f"{r['auc']:.4f} | {r['precision']:.4f} | {r['recall']:.4f} |")
    return "\n".join(lines)

if GAN_COMPLETE and MODEL_B is not None and METRICS:
    import importlib.metadata as metadata
    environment={"python":platform.python_version(),"tensorflow":tf.__version__,
        "keras":keras.__version__,"gpu":[tf.config.experimental.get_device_details(g).get("device_name",str(g)) for g in gpus],
        "platform":platform.platform(),"precision":POLICY,
        "packages":{p:metadata.version(p) for p in ["numpy","Pillow","scikit-learn","matplotlib","reportlab"]}}
    atomic_json(ROOT/"results"/"environment.json",environment)
    gan_history=load_json(ROOT/"results"/"srgan_history.json",[])
    fig,axes=plt.subplots(1,2,figsize=(11,3.5))
    adversarial_rows=[r for r in gan_history if r["phase"]=="adversarial"]
    for index,label in [(0,"Generator"),(1,"Discriminator")]:
        axes[0].plot([r["epoch"] for r in adversarial_rows],[r["losses"][index] for r in adversarial_rows],label=label)
    axes[0].set(xlabel="Adversarial epoch",ylabel="Loss");axes[0].legend()
    measured=[r for r in adversarial_rows if "sr_psnr" in r]
    axes[1].plot([r["epoch"] for r in measured],[r["sr_psnr"] for r in measured],label="SRGAN")
    axes[1].plot([r["epoch"] for r in measured],[r["bicubic_psnr"] for r in measured],label="Bicubic")
    axes[1].set(xlabel="Adversarial epoch",ylabel="Validation PSNR dB");axes[1].legend()
    save_figure("srgan_learning_curves.png")
    summary=f"""# SRGAN midterm cats versus dogs

## Reproduce the experiment
1. Open SRGAN_Midterm_Colab.ipynb in Google Colab and select a GPU runtime.
2. Obtain the SAME cats-versus-dogs dataset used in Assignment 1. Supply a ZIP or folder.
3. Mount Drive and set DATA_SOURCE, DRIVE_BASE, and EXPERIMENT_NAME in Section 2.
4. Run all sections in order. Do not change fixed settings while resuming.
5. After a disconnect, reconnect a GPU and run all again. Completed stages restore/skip.
6. Download the executed notebook, include these exported results, and provide dataset/model links.

## Dataset and split
Class 0 is cat; class 1 is dog. Dataset source citation/download link: ADD YOUR DATASET LINK.
Validated unique images: {len(Y):,}; corrupt/unknown exclusions: {DATA_META['rejected']}; duplicate removals: {DATA_META['duplica
Train pool is 70% and test is 30%, stratified with seed {CFG['seed']}.
Ten percent of the training pool is validation: {len(train_ids):,} fit, {len(val_ids):,} validation, {len(test_ids):,} test image
Manifest hashes and split indices are included. The old Assignment 1 split is rebuilt.
Images are RGB and resized to 128 x 128. GAN LR inputs are bicubic-antialiased 32 x 32.
Classification normalization is [0,1] then MobileNetV2 2x-1. Training augmentation:
horizontal flips, small rotations, zoom, contrast. Validation/test are unaugmented.

## Models and training
A trains on real HR images; B trains only on final SRGAN reconstructions, with inherited labels.
Both use the same ImageNet MobileNetV2 initialization and the same fitting image IDs.
```

```
Both train {CFG['head_epochs']} head epochs at 1e-3 and {CFG['fine_epochs']} fine-tuning epochs at 1e-5,
unfreezing the last 30 backbone layers except batch normalization. Lowest validation loss selects the classifier.
SRGAN uses {CFG['g_blocks']} residual blocks with {CFG['g_filters']} channels and two pixel-shuffle stages.
This is a compact adaptation, not the exact paper architecture. VGG19 {CFG['vgg_layer']} is frozen.
Generator loss is L1 + 0.01*MSE(VGG/12.75) + 0.001*adversarial BCE. D uses real target 0.9 and fake target 0.
Completed warm-up epochs: {int(warmup_done.numpy())}; completed full adversarial epochs: {int(gan_done.numpy())}.
Warm-up is additional to the required 150 adversarial epochs. All fitting images are visited in every epoch.
Adam learning rate is 1e-4 for G/D. Batch size is {CFG['gan_batch']}; policy is {POLICY}.
Final-epoch G, rather than a test-selected generator, supplies B. Full-state checkpoints save every {CFG['checkpoint_every']} epo
Random seeds are fixed, but resumed GPU training is not guaranteed bitwise identical.

## Classifier results
Primary comparison uses the identical REAL test images for A and B. Secondary conditions apply
the identical bicubic or SRGAN inputs to both. Dog is positive. Threshold is 0.5; ROC AUC uses probabilities.

{markdown_table(METRICS)}

## Reconstruction results
Full held-out test set, [0,1] RGB: {json.dumps(TEST_QUALITY)}.
PSNR is dB (capped at 100 for exact matches); SSIM is unitless. Both are averaged per image, with no border crop.
SRGAN may invent plausible texture. Improved appearance does not guarantee higher accuracy or fidelity.

## Environment and files
TensorFlow {tf.__version__}; Keras {keras.__version__}; Python {platform.python_version()}.
GPU: {environment['gpu']}. See results/environment.json and experiment_spec.json for exact settings.
Checkpoints and large arrays are excluded from the GitHub bundle. Model/download links: ADD YOUR MODEL LINKS.
The exported ZIP does not automatically include the executed Colab notebook. Download and add it separately.

## Discussion to complete after reviewing the figures
Explain whether B improved accuracy, F1, or AUC on the common real test set, and by how much.
Discuss behavior on bicubic/SRGAN inputs, visible texture artifacts, and PSNR/SSIM versus classification.
Report the actual GPU and elapsed training times from the logs. Do not reuse Assignment 1's scores.

## References
- Ledig et al. Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network. CVPR 2017. https://arxiv.org
- Keras MobileNetV2 https://keras.io/api/applications/mobilenet/
- TensorFlow checkpoints https://www.tensorflow.org/guide/checkpoint
- Keras mixed precision https://keras.io/api/mixed_precision/
"""
    (ROOT/"README.md").write_text(summary,encoding="utf-8")
    # Reproducibility PDF: actual results and key procedure, with no invented scores.
    from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle, PageBreak, Image as PDFImage
    from reportlab.lib.styles import getSampleStyleSheet
    from reportlab.lib import colors
    from reportlab.lib.units import inch
    from reportlab.pdfbase import pdfmetrics
    from reportlab.pdfbase.ttfonts import TTFont
    from matplotlib.font_manager import findfont, FontProperties
    from xml.sax.saxutils import escape
    pdfmetrics.registerFont(TTFont("ReportSans",findfont("DejaVu Sans")))
    pdfmetrics.registerFont(TTFont("ReportSans-Bold",findfont(FontProperties(family="DejaVu Sans",weight="bold"))))
    pdfmetrics.registerFontFamily("ReportSans",normal="ReportSans",bold="ReportSans-Bold",
                                  italic="ReportSans",boldItalic="ReportSans-Bold")
    styles=getSampleStyleSheet()
    for style_name in styles.byName:
        styles[style_name].fontName="ReportSans-Bold" if style_name.startswith("Heading") or style_name=="Title" else "ReportSans
    story=[Paragraph("SRGAN Midterm Experiment Report",styles["Title"]),Spacer(1,12)]
    for title,body in [
        ("Purpose", "Compare cats-versus-dogs classifiers trained on real images and on learned 4x super-resolution reconstructio
        ("Reproduction", "Open the supplied notebook in Colab, select a GPU, mount Drive, set DATA_SOURCE to the Assignment 1 da
        ("Data and split", f"{len(Y):,} unique RGB images. Seed {CFG['seed']}. Outer split 70 percent training pool and 30 percen
        ("Image preparation", "Targets are 128 x 128 RGB. LR inputs are 32 x 32 using antialiased bicubic downsampling. GAN value
        ("Classifiers", f"A uses real targets. B uses only generated fitting images. Both start from identical ImageNet MobileNet
        ("SRGAN", f"Compact adaptation: {CFG['g_blocks']} residual blocks, {CFG['g_filters']} channels, two pixel-shuffle stages
        ("Evaluation", "The primary A/B comparison uses the same real test images. Both are also evaluated on identical bicubic a
        ("Resume and runtime", f"Checkpoints include G/D, both optimizers, loss scales and epoch counters. They are saved every
    ]:
        story += [Paragraph(title,styles["Heading2"]),Paragraph(escape(body),styles["BodyText"]),Spacer(1,6)]
    table_data=[["Model","Test input","Accuracy","F1","AUC"]]+[[r["model"],r["test_input"],f"{r['accuracy']:.4f}",f"{r['f1']:.4f
    table=Table(table_data,colWidths=[55,100,85,85,85],repeatRows=1)
    table.setStyle(TableStyle([("FONTNAME",(0,0),(-1,-1),"ReportSans"),("BACKGROUND",(0,0),(-1,0),colors.HexColor("#183348")),("
    quality_data=[["Reconstruction","PSNR dB","SSIM"],
                  ["Bicubic",f"{TEST_QUALITY['bicubic_psnr']:.3f}",f"{TEST_QUALITY['bicubic_ssim']:.4f}"],
                  ["SRGAN",f"{TEST_QUALITY['sr_psnr']:.3f}",f"{TEST_QUALITY['sr_ssim']:.4f}"]]
    quality_table=Table(quality_data,colWidths=[150,130,130],repeatRows=1)
    quality_table.setStyle(TableStyle([("FONTNAME",(0,0),(-1,-1),"ReportSans"),
        ("BACKGROUND",(0,0),(-1,0),colors.HexColor("#e9eff4")),
```

```
        ('TOPPADDING',(0,0),(-1,-1),7),('BOTTOMPADDING',(0,0),(-1,-1),7)]))
    story += [PageBreak(),Paragraph("Measured classifier scores",styles["Heading2"]),table,
```